

Search System — Practical System Design Notes

Hinglish • Inverted Index • Elasticsearch/OpenSearch • Ranking • Autocomplete • Scaling

1. Search System Kya Hai?

Search system ka goal hai user ke query ke basis par large dataset mein relevant results ko quickly find aur rank karna. E-commerce product search, job search, document search aur website search iske common examples hain.

```
User
↓
Search API
↓
Search Engine
↓
Index
↓
Ranked Results
```

2. Database Search vs Search Engine

Database	Search Engine
Primary data storage	Search/retrieval optimized index
Exact/filter queries strong	Full-text/relevance search strong
Transactions/source of truth	Fast text retrieval and ranking
Example: MySQL/PostgreSQL	Elasticsearch/OpenSearch

Search engine generally source of truth nahi hota. Application data primary DB mein store hota hai aur search index uska optimized searchable representation hota hai.

3. Why LIKE Can Become Problematic

```
SELECT *
FROM products
WHERE name LIKE '%iphone%';
```

```
Large table
↓
many rows scanned
↓
latency increases
```

Small datasets ke liye DB full-text features useful ho sakte hain, but large-scale relevance search, typo tolerance, ranking and autocomplete ke liye dedicated search engine better fit ho sakta hai.

4. Inverted Index — Most Important

Inverted index word ko documents ki list se map karta hai. Ye search engine ki core idea hai.

Documents:

```
D1: "red running shoes"
D2: "blue running shoes"
D3: "red jacket"
```

Inverted Index:

```
red    → D1, D3
running→ D1, D2
shoes  → D1, D2
blue   → D2
jacket → D3
```

5. Why Inverted Index Fast Hai?

Query 'red shoes' ke liye search engine relevant term postings ko directly access kar sakta hai, instead of every document ka complete text scan karne ke.

Query: "red shoes"

red postings



D1, D3

shoes postings



D1, D2

Intersection / scoring



D1 highest match

6. Tokenization

Search engine text ko tokens/terms mein break karta hai.

"Red Running Shoes"

→ red

→ running

→ shoes

7. Normalization

Case, punctuation, accents aur other text normalization query quality improve kar sakti hai.

"iPhone, iPHONE"

normalize



"iphone"

Both can match consistently.

8. Stemming / Lemmatization

Related word forms ko common representation mein map karne ki techniques search recall improve kar sakti hain. Exact behavior language aur analyzer configuration par depend karta hai.

running

runner

runs

possible search analysis

→ run

9. Search Indexing Pipeline

```
Primary DB
  ↓
CDC / Event / Queue
  ↓
Indexer Worker
  ↓
Search Engine Index
  ↓
Search API
```

DB aur search index ko loosely coupled rakhne ke liye asynchronous indexing common architecture hai.

10. CDC — Change Data Capture

CDC database changes ko capture karke downstream systems tak propagate kar sakta hai. Search index ko source DB ke changes ke saath update karne ke liye CDC useful approach hai.

```
DB INSERT/UPDATE
  ↓
CDC
  ↓
Event
  ↓
Indexer
  ↓
Search Index
```

11. Queue-Based Indexing

```
Product Service
  ↓
DB
  ↓
Outbox / Event
  ↓
Queue
  ↓
Search Indexer
  ↓
Elasticsearch/OpenSearch
```

Queue temporary spikes absorb karti hai aur indexing workers ko independently scale karne deti hai.

12. Eventual Consistency

DB update ke immediately baad search index update na bhi hua ho sakta hai. Is gap ko eventual consistency kehte hain.

t0:
DB = new product

t1:
Search index = old

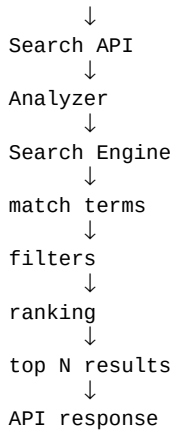
t2:
Indexer runs

t3:
Search index = new

Business requirements ke according indexing delay acceptable hai ya synchronous update required hai, ye decide karna hota hai.

13. Search Query Flow

User:
"red nike shoes"



14. Ranking / Relevance

Search engine multiple signals ke basis par results ko score/rank kar sakta hai. Text relevance, field importance, exact match, popularity, freshness or business rules combine kiye ja sakte hain.

Query:
"nike shoes"

Result A:
exact phrase + strong title match
→ high score

Result B:
only description match
→ lower score

15. Field Boosting

title^3
name^2
description^1

Query match in title
→ more important

Query match in description
→ less important

Exact weights product/search requirements ke according tune kiye jaate hain.

16. Filters

Search:
"running shoes"

Filters:
brand = Nike
size = 9
price < 5000
rating >= 4

Query
+
Filters
↓
Results

Filters usually relevance query se separate dimensions hote hain. Faceted search mein filters ke available counts bhi return kiye ja sakte hain.

17. Faceted Search

Products

Brand:

Nike (120)

Adidas (95)

Puma (70)

Size:

8 (80)

9 (110)

10 (75)

Price:

< 2000

2000-5000

> 5000

E-commerce search mein facets user ko result set narrow karne mein help karte hain.

18. Autocomplete

User types:

"iph"

Suggestions:

iphone

iphone 15

iphone 15 pro

iphone charger

Autocomplete ko low latency chahiye. Prefix indexes, tries, edge-ngram style analyzers, specialized suggestion structures ya cached popular queries use kiye ja sakte hain.

19. Typo Tolerance

Query:

"iphnoe"

Expected:

"iphone"

Possible techniques:

fuzzy matching

edit distance

spell correction

dictionary/popularity signals

20. Edit Distance

Edit distance roughly measure karta hai kitne insert/delete/substitute operations se ek string ko doosri string mein convert kiya ja sakta hai.

iphone

iphnoe

small edit distance

→ likely typo

21. Search Cache

Query:

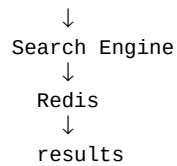
"nike shoes"

↓

Redis

■ → HIT → results

■ → MISS



Repeated popular queries cache ki ja sakti hain, but cache key mein query + filters + sort + page/cursor + relevant user/tenant context carefully include karna padta hai.

22. Cache Invalidation

Search results data changes frequently. TTL-based cache, versioned keys, event-driven invalidation or short cache windows common approaches hain.

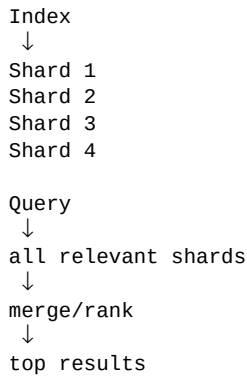
23. Pagination

GET /search?q=iphone&limit=20

For large result sets:
cursor/search_after
rather than huge offset pagination

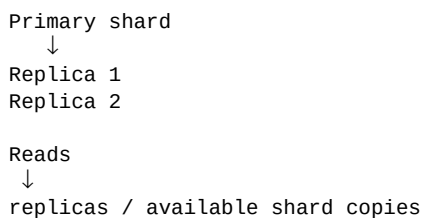
Deep pagination search engine resources expensive kar sakti hai. Cursor/search_after style approaches large result sets ke liye better fit ho sakte hain.

24. Search Index Shards



Sharding index ko horizontally distribute karta hai. More shards always better nahi hote; shard count operational and query overhead create karta hai.

25. Search Replicas

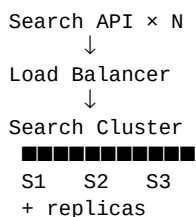


Replicas availability aur read capacity improve kar sakte hain. Exact behavior search platform and deployment architecture par depend karta hai.

26. Hot Shard / Data Skew

Poor shard distribution se ek shard par disproportionately high traffic/data aa sakta hai. Shard key and index design carefully choose karo.

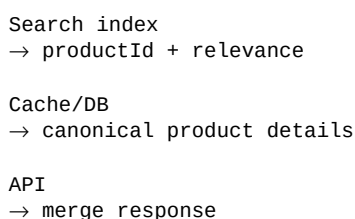
27. Search Cluster Scaling



API servers stateless rakhna aur search cluster ko independently scale karna operationally useful hota hai.

28. Search Result Hydration

Search index mein complete product object rakhne ke bajay lightweight searchable fields + document ID store ki ja sakti hai. API DB/cache se canonical details fetch karke hydrate kar sakti hai, but this adds latency and N+1 risk; trade-off evaluate karo.



29. Denormalization

Search engine mein denormalized document store karna queries ko fast bana sakta hai. Example: product document mein brand name, category name and searchable attributes embedded ho sakte hain.

30. Index Update Strategy

```
Product update
↓
event
↓
queue
↓
indexer
↓
upsert search document
```

Indexer ko idempotent banana useful hai so duplicate events se incorrect state na bane.

31. Full Reindex

Kabhi analyzer/index mapping change hone par complete reindex required ho sakta hai.

Old Index
products_v1

Build:
products_v2
↓
bulk reindex
↓
validate
↓
switch alias
↓
products_v2 active

Blue/green style index replacement se rebuild ke dauran search availability maintain ki ja sakti hai, depending on platform.

32. Bulk Indexing

1 million products

Instead of:
1 request × 1 million

Use:
bulk batches
↓
indexer workers
↓
Search cluster

Bulk operations network and indexing overhead reduce kar sakte hain, but batch size and concurrency tune karni hoti hai.

33. Search Analytics

- Top search queries.
- Zero-result queries.
- Click-through rate.
- Search-to-purchase conversion.
- Autocomplete selection rate.
- Average search latency.
- p95/p99 latency.
- Query error rate.

34. Zero Result Queries

Query:
"red xyz shoes"

Results:
0

Analytics event
↓
Search improvement pipeline
↓
synonyms / catalog / ranking improvements

Zero-result queries product/search quality improve karne ke liye valuable signal hain.

35. Search Suggestions from Analytics

Popular queries, click behavior and zero-result terms se suggestion dictionaries/ranking improve ki ja sakti hai. Analytics processing ko search request path se separate rakho.

36. Search Security

- Authenticate private/tenant search.
- Apply tenant-level filtering.
- Avoid exposing documents a user is not authorized to see.
- Rate-limit expensive queries.
- Restrict query complexity where necessary.
- Do not blindly expose raw search engine APIs to public clients.

37. Multi-Tenant Search

Tenant A
↓
Index / filtered documents

Tenant B
↓
Index / filtered documents

Query must include:
tenantId / authorization filter

Authorization filter search query ka mandatory part ho sakta hai. Security boundary ko sirf UI filters par depend mat karo.

38. Search vs Cache

Cache	Search Engine
Stores already computed/retrieved results	Finds/ranks matching documents
Fast exact key lookup	Text and relevance queries
TTL/invalidation important	Indexing/mapping/sharding important

39. Failure Scenarios

Failure	Handling
Search cluster down	Fallback to simpler DB search/cache where feasible, or degrade gracefully
Indexer down	Queue backlog grows; recover workers
Index lag	Monitor lag and accept/handle eventual consistency
Hot query	Cache/query optimization/rate limiting
Hot shard	Rebalance/shard strategy/index design
Bad index mapping	Build corrected index and switch
DB updated but index stale	Retry/event replay/reindex
Zero results	Synonyms/fuzzy/catalog/ranking improvements

40. Production Architecture

Users
↓
CDN / WAF (optional)
↓
Load Balancer
↓
Search API × N
↓
Redis Query Cache (optional)
↓
Elasticsearch / OpenSearch Cluster
 ■■■ Primary shards
 ■■■ Replica shards

Indexing:
Primary DB
↓
CDC / Outbox
↓
Queue
↓
Indexer Workers
↓

Search Cluster

Analytics:

Search events



Queue



Analytics pipeline

41. End-to-End Indexing Flow

1. Product created/updated
2. DB commits
3. Event/outbox created
4. Queue receives event
5. Indexer consumes event
6. Transform/normalize document
7. Upsert search index
8. Record processing success
9. Monitor indexing lag

42. End-to-End Search Flow

1. User enters query
2. API authenticates/authorizes
3. Normalize query
4. Check query cache
5. Search engine query
6. Apply filters
7. Rank results
8. Return top N
9. Publish analytics event asynchronously

43. Relevance Tuning

- Title/name match boost.
- Exact phrase boost.
- Popularity signal.
- Freshness signal.
- Business rules.
- Availability/in-stock filtering.
- Personalization where appropriate.
- Synonyms.

Ranking should be measurable. Offline evaluation sets, click metrics and business metrics can be used to compare search changes.

44. Synonyms

Query:
"mobile"

Synonyms:
phone
smartphone

Search:
mobile
↓
mobile OR phone OR smartphone

Synonym rules carefully manage karo; aggressive synonyms irrelevant results increase kar sakte hain.

45. Autocomplete Architecture

User types "iph"
↓
Autocomplete API
↓
Redis / Search suggestion index
↓

top suggestions



Client

Autocomplete ke liye very low latency important hai. Query cache and precomputed popular prefixes useful ho sakte hain.

46. Local Node.js Practice

Node.js + Express + Elasticsearch/OpenSearch + Redis + PostgreSQL/MySQL use karke product search banao. DB ko source of truth rakho, indexing queue/outbox banao, search index mein denormalized product documents rakho, filters, autocomplete, fuzzy search, analytics and Grafana metrics implement karo.

Phase 1:

DB products



Search API

Phase 2:

Index products

Phase 3:

Query + filters

Phase 4:

Redis cache

Phase 5:

Queue/Outbox

→ async indexing

Phase 6:

Autocomplete + fuzzy

Phase 7:

Ranking

Phase 8:

Analytics

Phase 9:

3 API instances + monitoring

47. Interview Q&A;

Question	Short answer
Search engine kyun?	Full-text retrieval, relevance, filters and autocomplete at scale.
Inverted index kya?	Term → documents/postings mapping for fast text retrieval.
DB vs search engine?	DB source of truth; search engine optimized searchable index.
Eventual consistency?	DB update aur index update ke beech temporary lag.
CDC?	Database changes capture karke downstream index update karna.
Redis kyun?	Popular search queries/autocomplete caching ke liye low-latency cache.
Autocomplete?	Prefix/suggestion optimized index or cache with low latency.
Typo tolerance?	Fuzzy matching/edit distance/spell correction.
Ranking?	Relevance + field boosts + business/freshness/popularity signals.
Shards?	Index data ko horizontally distribute karna.
Replicas?	Availability and read capacity improve karna.
Reindex?	New mapping/analyzer/index build karke bulk data repopulate karna.
Index lag?	Queue/worker lag monitor, retry/replay and reindex when needed.

48. Practice Scenarios

1. Design product search for 100 million products.
2. Explain inverted index with a simple example.
3. DB product update hua but search result old hai. Why?
4. How will you reduce search latency?
5. Design autocomplete for 1 million queries/day.
6. User types a misspelled product name. How do you handle it?
7. Design filters for e-commerce search.
8. Search cluster ka ek shard hot ho gaya. What do you inspect?
9. Search index corrupt/outdated ho gaya. How do you rebuild it?
10. How do you make private/tenant search secure?
11. Design zero-result query analytics.
12. Explain why search indexing is usually asynchronous.

49. Final Cheat Sheet

SEARCH SYSTEM

SOURCE OF TRUTH:

DB
↓
CDC / Outbox
↓
Queue
↓
Indexer
↓
Search Index

QUERY:

User

↓
Search API
↓
Cache
↓
Search Engine
↓
Filters
↓
Ranking
↓
Top Results

Core concepts:
→ inverted index
→ tokenization
→ normalization
→ analyzers
→ ranking
→ filters/facets
→ autocomplete
→ fuzzy search
→ caching
→ eventual consistency
→ CDC/outbox
→ shards
→ replicas
→ reindexing
→ search analytics

Golden interview line:
"I'll keep the primary DB as the source of truth and build an asynchronous search index through an outbox/CDC pipeline. Search APIs query the distributed index with filters and relevance ranking, while Redis can cache hot queries. I'll monitor index lag, shard health, p95/p99 latency and zero-result queries."